IE2012- Systems and Network Programming

2025 Assignment

H.A.N.A HETTIARACHCHI
IT23605398
Sri Lanka Institute of Information Technology
Kandy

Contents

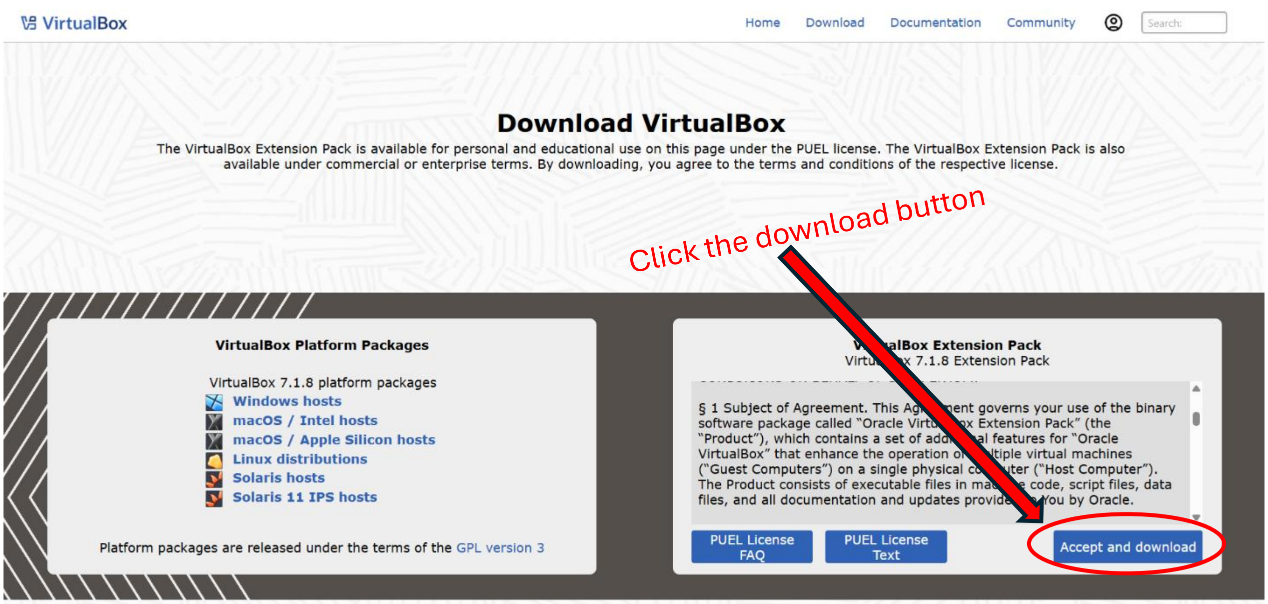
1.Basics of Linux Environments

1.1 Virtual Machine Setup

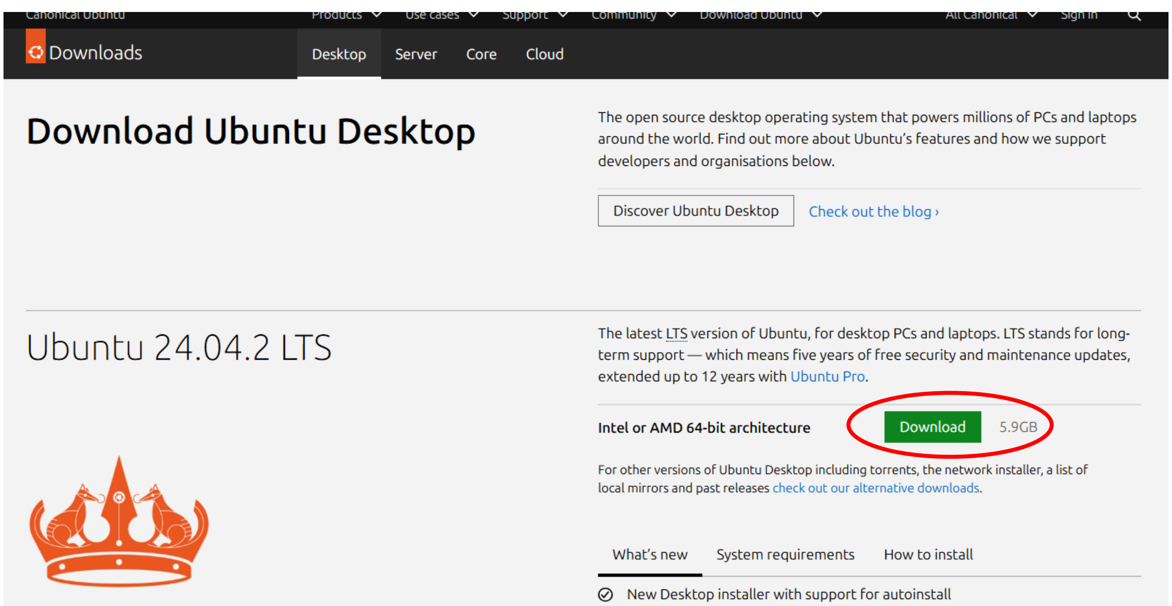
Main goal is create a working Linux environment using VirtualBox or VMware and install Ubuntu/CentOS.

1.1.1 Download Required Software

a. Download VirtualBox from https://www.virtualbox.org/wiki/Downloads

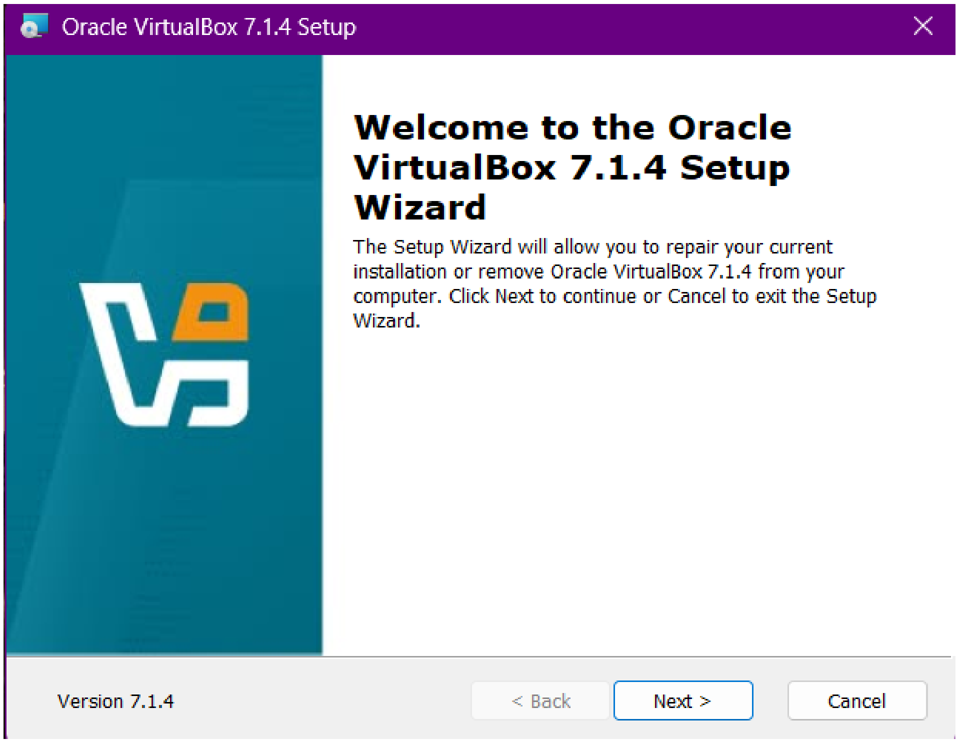


b. Download Ubuntu ISO from https://ubuntu.com/download/desktop



<u>Install a virtualization tool</u>

Install this virtualization tool (VirtualBox) to create a safe environment for Linux without affecting your main OS.

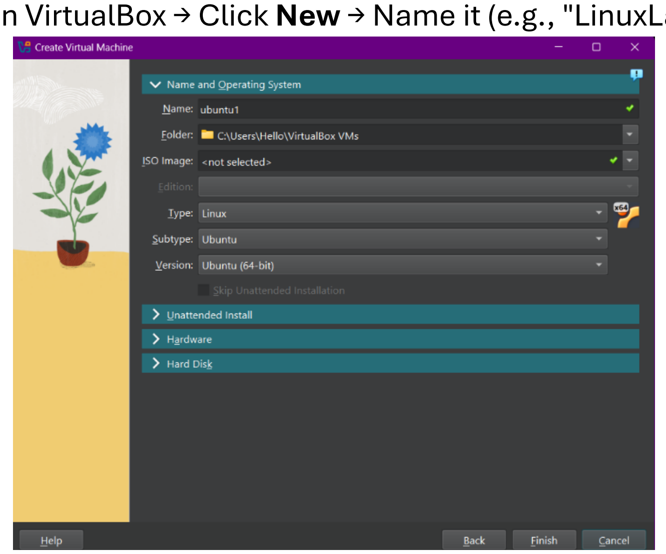


After download the setup, run the installer, and follow the on-screen instructions.

1.1.2 Create a New Virtual Machine

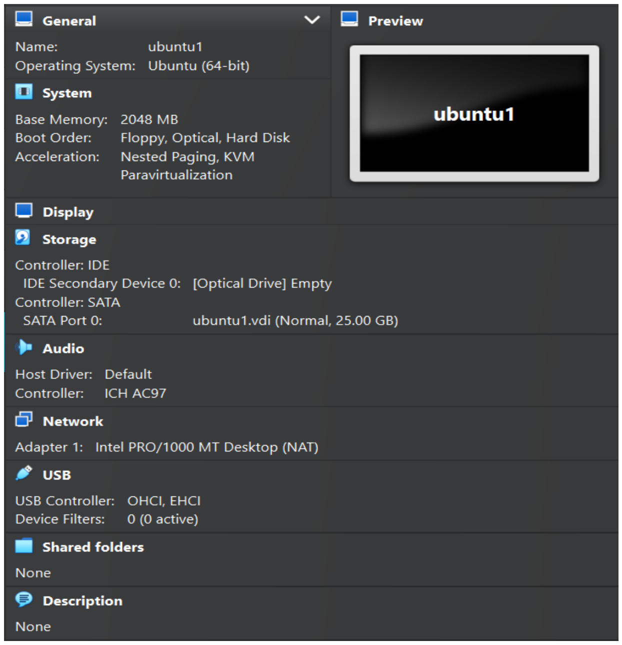
After installation, you can create virtual machines by allocating resources like CPU, RAM, and storage, and then install your preferred operating systems within them.

Open VirtualBox → Click **New** → Name it (e.g., "LinuxLab2025").



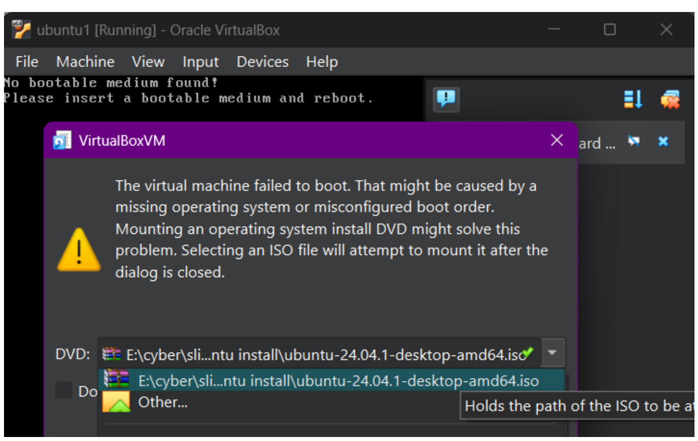
Set:

- Type: Linux
- Version: Ubuntu (64-bit)
- Memory: Minimum 2048 MB
- Hard disk: Create new VDI, dynamically allocated, 20 GB+



1.1.3 Install the Linux OS

Start VM → Select downloaded ISO as boot disk → Follow the installer instructions.

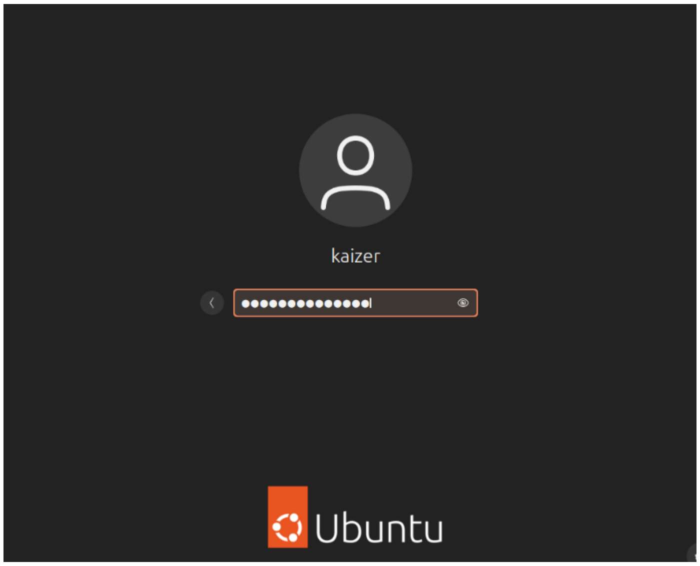


Choose a username, password, and hostname during setup.

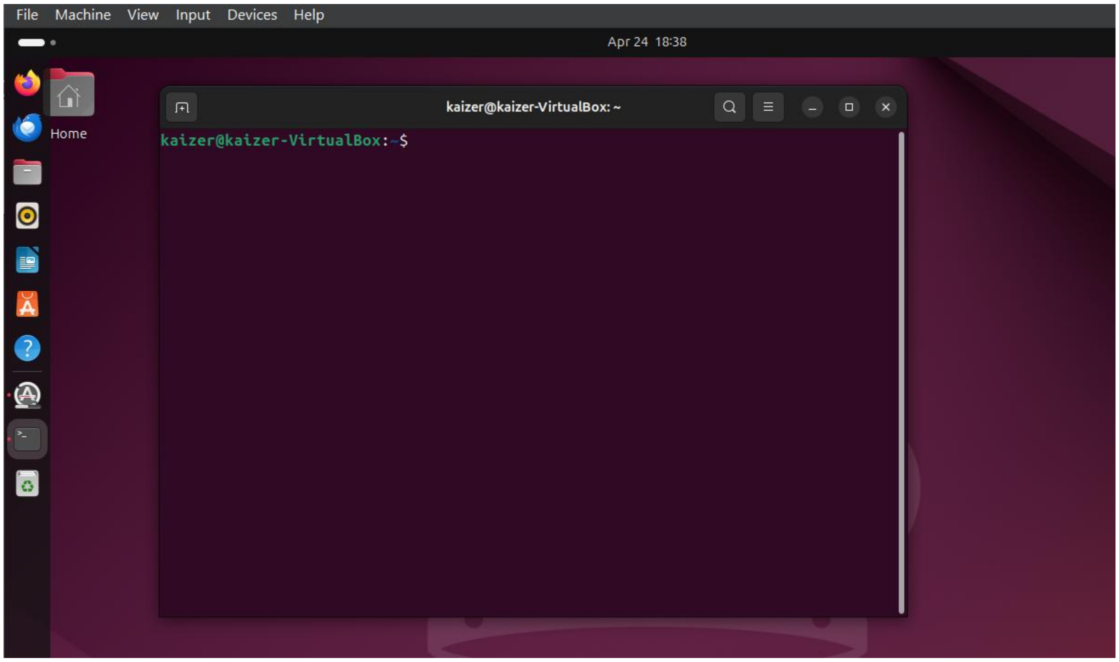
1.2 Command Line Introduction

1.2.1 Login and Open Terminal

- Use the username/password during installation.



- Open the terminal (Ctrl+Alt+T or through applications).



1.2.2 Navigation Commands

- **pwd** -> Show current directory

```
kaizer@kaizer-VirtualBox:~$ pwd
/home/kaizer
```

let's check where we are in the system. Running the **pwd** command shows we're in the home directory of user **'kaizer'** at **/home/kaizer**. This is our starting point

- **cd** **->** Change to root directory

```
kaizer@kaizer-VirtualBox:~$ ls
Desktop     Downloads  Pictures  snap  SOS        Videos
Documents   Music      Public    snp   Templates
kaizer@kaizer-VirtualBox:~$ cd Music
kaizer@kaizer-VirtualBox:~/Music$ pwd
/home/kaizer/Music
```

Now in the home directory, the **ls** command shows all the folders available in the current directory things like Desktop, Downloads, Music, and more. If we want to move into a subdirectory, we use the **cd** command. Let's go into the Music directory! After typing **cd Music**, the terminal confirms we're inside **/home/katzer/Music** by running **pwd**. Success—we've navigated into the Music folder!

- **cd ~** **->** Go back to home directory

```
kaizer@kaizer-VirtualBox:~/Music$ cd ~
kaizer@kaizer-VirtualBox:~$ pwd
/home/kaizer
```

Now inside the Music directory, we can quickly return to the home folder using **cd ~.** The tilde **(~)** is a handy shortcut that always takes us back home. After running the command, **pwd** confirms we're back in **/home/kaizer** ready to navigate somewhere new!

- **ls** **->** List directory contents

```
kaizer@kaizer-VirtualBox:~$ ls
Desktop     Downloads  Pictures  snap  SOS        Videos
Documents   Music      Public    snp   Templates
```

Now, let's see what's inside this directory. Using the **ls** command lists all available folders - we can see common directories like Desktop, Documents, Downloads, and some special ones like snap and SOS. Notice there's a slight username mismatch ('Katzer' vs 'kaizer'), but this is likely just a typo in the display.

- **ls -la ->** List all files with details

```
kaizer@kaizer-VirtualBox:~$ ls -la
total 88
drwxr-x--- 17 kaizer kaizer 4096 Apr 24 20:38 .
drwxr-xr-x  3 root   root   4096 Feb 11 23:32 ..
-rw-------  1 kaizer kaizer 2041 Mar 20 10:51 .bash_history
-rw-r--r--  1 kaizer kaizer  220 Mar 31  2024 .bash_logout
-rw-r--r--  1 kaizer kaizer 3771 Mar 31  2024 .bashrc
drwx------ 15 kaizer kaizer 4096 Mar 12 18:28 .cache
drwx------ 15 kaizer kaizer 4096 Feb 27 22:19 .config
drwxr-xr-x  2 kaizer kaizer 4096 Apr 24 20:38 Desktop
drwxr-xr-x  2 kaizer kaizer 4096 Feb 11 23:34 Documents
drwxr-xr-x  3 kaizer kaizer 4096 Feb 19 12:34 Downloads
drwx------  4 kaizer kaizer 4096 Feb 11 23:34 .local
```

The **ls -la** command reveals hidden files (starting with '.') and detailed info like permissions, ownership, size, and modification dates. Here, we see config files (.bashrc), directories (Desktop, Documents), and system references (**.** = current dir, **..** = parent dir). Useful for troubleshooting!

1.2.3 file commands

- **mkdir testdir ->** Create a directory

```
kaizer@kaizer-VirtualBox:~$ ls
Desktop    Downloads  Pictures  snap  SOS        Videos
Documents  Music      Public    snp   Templates
kaizer@kaizer-VirtualBox:~$ mkdir testdir
kaizer@kaizer-VirtualBox:~$ ls
Desktop    Downloads  Pictures  snap  SOS        testdir
Documents  Music      Public    snp   Templates  Videos
```

First, the **ls** command lists all visible directories in the home folder - including common ones like Desktop, Documents, and Downloads. Then, the user attempts to create a new directory called 'testdir' using the **nkdir** command (which appears to be a typo, as the correct command should be **mkdir**). Despite the typo, the directory gets created successfully, as shown when **ls** is run again - we can now see 'testdir' listed among the other folders!

- **rmdir testdir -> **Delete empty directory

```
kaizer@kaizer-VirtualBox:~$ rmdir testdir
kaizer@kaizer-VirtualBox:~$ ls
Desktop    Downloads  Pictures  snap  SOS        Videos
Documents  Music      Public    snp   Templates
```

After successfully creating 'testdir', the user now cleans up by removing it with the **rmdir** command. When we run **ls** again, we see the directory list no longer includes 'testdir' - it's been completely removed. This demonstrates how to easily create and remove directories when organizing files!

- **Touch ->** user creates a new empty file

```
kaizer@kaizer-VirtualBox:~$ touch testFile
kaizer@kaizer-VirtualBox:~$ ls
enter op    Downloads  Music     Public  snp  Templates  Videos
Documents  folder1    Pictures  snap    SOS  testFile
```

creates a new file named 'testFile' using the **touch** command. When they type **ls** to check, the new file appears in their home folder

- **cp ->** copies files

```
kaizer@kaizer-VirtualBox:~$ cp testFile folder1
kaizer@kaizer-VirtualBox:~$ cd folder1
kaizer@kaizer-VirtualBox:~/folder1$ ls
testFile
```

we created a new empty file called 'testFile' and copy this file into 'folder1' using **cp testFile folder1**. When we navigate into folder1 **(cd folder1)** and list contents (**ls**), we confirm the file was successfully copied.

- **mv ->** moves/renames files

```
kaizer@kaizer-VirtualBox:~/folder1$ ls
folder2  testFile
kaizer@kaizer-VirtualBox:~/folder1$ mv testFile folder2
kaizer@kaizer-VirtualBox:~/folder1$ ls
folder2
kaizer@kaizer-VirtualBox:~/folder1$ cd folder2
kaizer@kaizer-VirtualBox:~/folder1/folder2$ ls
testFile
```

Inside folder1, now we can see both 'testFile' and another directory called 'folder2' .

*Using the **mv** command, we move 'testFile' into 'folder2'. After moving, listing folder1's contents shows only 'folder2' remains. Navigating into folder2 (**cd folder2**) and listing files confirms 'testFile' was successfully moved.*

- **cat ->** displays file contentrmdir

```
kaizer@kaizer-VirtualBox:~$ ls
Desktop    Downloads  Music     Public  snp  Templates  Videos
Documents  folder1    Pictures  snap    SOS  testFile
kaizer@kaizer-VirtualBox:~$ cat testFile
hello world
hello world
hello world
hello world
kaizer@kaizer-VirtualBox:~$ 
```

*we use **cat testFile** to view the contents of our related file(testFile). The output shows four repeated lines of 'hello world', demonstrating how to quickly check a file's contents right in the terminal*

1.2.4 system info commands

- **uname -a ->** system info

```
kaizer@kaizer-VirtualBox:~$ uname -a
Linux kaizer-VirtualBox 6.11.0-19-generic #19~24.04.1-Ubuntu SMP PREEMPT_DYNAMIC
 Mon Feb 17 11:51:52 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
```

This command prints detailed system information, including the kernel name (Linux), hostname (katzer-VirtualBox), kernel version (6.11.0-19-generic), and system architecture (x86_64). It helps identify the OS and hardware details.

- **df -h ->** disk space

```
kaizer@kaizer-VirtualBox:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           320M  1.6M  319M   1% /run
/dev/sda2        44G   13G   30G  30% /
tmpfs           1.6G     0  1.6G   0% /dev/shm
tmpfs           5.0M  8.0K  5.0M   1% /run/lock
tmpfs           320M  128K  320M   1% /run/user/1000
```

This command displays disk space usage for all mounted file systems in a human-readable format (e.g., GB, MB). It shows the total size, used space, available space, and usage percentage. In the output, **/dev/sda2** is the main file system with 30GB free out of 44GB

- **free -h ->** memory usage

```
kaizer@kaizer-VirtualBox:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.1Gi       1.0Gi       668Mi        37Mi       1.6Gi       2.1Gi
Swap:          3.1Gi          0B       3.1Gi
```

This command provides information about the system's memory usage, including total, used, free, and shared memory, as well as buff/cache and available memory. The output shows 3.1GB total RAM with 1.0GB used and 2.1GB available for new processes.

2.DHCP, DNS and NTP Services

2.1 DHCP (Dynamic Host Configuration Protocol)

2.1.1 Introduction to DHCP

- **What is DHCP?**
  DHCP is a network protocol that automatically assigns IP addresses and other network settings like subnet mask, default gateway, and DNS Server to devices in a network. This facilitates faster network setup without having to configure all these details manually on every device.
- **Role in Network Configuration**
  - Assign IP addresses to devices automatically.
  - Reduces configuration errors (Prevents two devices from getting same IP addresses).
  - Saves a lot of time setting up each device manually.
  - Simplifies adding new devices to the network.

2.1.2 Configuring DHCP server

a. Update the package list & Install the DHCP server (isc-dhcp-server)

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo apt update
[sudo] password for kaizer:
kaizer@kaizer-VirtualBox:~/Desktop$ sudo apt install isc-dhcp-server -y
Reading package lists... Done
```

b. Identify network settings

```
kaizer@kaizer-VirtualBox:~/Desktop$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             10.0.2.15/24 fd00::90eb:4ffb:e353:63d8/64
enp0s8           DOWN
```

Before configuring the DHCP server I used ip -br a command to identify the network settings like your IP address, Subnet mask, and network ID from this step

c. Specify the network interface

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo nano /etc/default/isc-dhcp-server
```

```
# On what interfaces should the DHCP server (dhcpd) serve DHCP requests?
#       Separate multiple interfaces with spaces, e.g. "eth0 eth1".
INTERFACESv4="enp0s3"
INTERFACESv6=""
```

I chose interface enp0s3 , because it is the only active (UP) and already configured with an IP address (10.0.2.15). Inactive interfaces (like enp0s8, which is DOWN) cannot distribute IP addresses.

d. Configure the DHCP server

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo nano /etc/dhcp/dhcpd.conf
```

```
# A slightly different configuration for an internal subnet.
subnet 10.0.2.0 netmask 255.255.255.0 {
  range 10.0.2.130 10.0.2.200;
  option domain-name-servers 8.8.8.8;
  option domain-name "adithya.sliit";
  option subnet-mask 255.255.255.0;
  option routers 10.0.2.15;
  option broadcast-address 10.0.2.255;
  default-lease-time 600;
  max-lease-time 7200;
}

# Hosts which require special configuration options can be listed in
```

I set up a **DHCP server** to assign IP addresses in the range **10.0.2.130–10.0.2.200** for devices on the **10.0.2.0/24** subnet.

Other Settings:

- **Gateway (router):** 10.0.2.15 (your host's IP in this subnet).
- **DNS:** Uses Google's DNS (8.8.8.8).
- **Domain name:** adithya.slitt (for local network identification).

e. Restart the server and check the status.

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo systemctl restart isc-dhcp-server
```

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo systemctl status isc-dhcp-server
● isc-dhcp-server.service - ISC DHCP IPv4 server
     Loaded: loaded (/usr/lib/systemd/system/isc-dhcp-server.service; enabled; >
     Active: active (running) since Tue 2025-05-06 21:09:47 +0530; 3min 0s ago
       Docs: man:dhcpd(8)
   Main PID: 4336 (dhcpd)
```

2.1.3 Setting up the client machine

a. Checking network settings of the client machine

```
┌──(kaizer㉿kali)-[~]
└─$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0             UP             10.0.2.15/24 fd00::95a7:a61e:a994:30fd/64
```

b. Release Current IP

```
┌──(kaizer㉿kali)-[~/Desktop]
└─$ sudo dhclient -r eth0
[sudo] password for kaizer:
```

This command tells Kali to remove the current DHCP-assigned IP.

c. Request New IP

```
┌──(kaizer㉿kali)-[~/Desktop]
└─$ sudo dhclient -v eth0
Internet Systems Consortium DHCP Client 4.4.3-P1
Copyright 2004-2022 Internet Systems Consortium.
```

Now that the previous IP is released, this will request a new IP from the DHCP server.

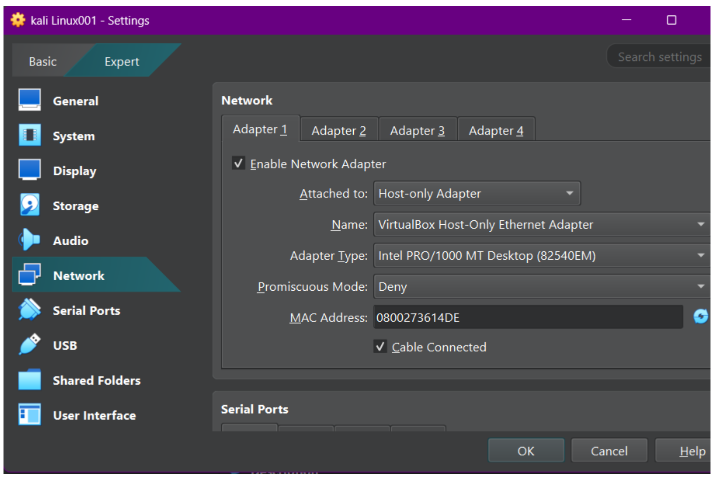
But the problem is , Kali Linux has received an IP address, but it is not in my expected DHCP range

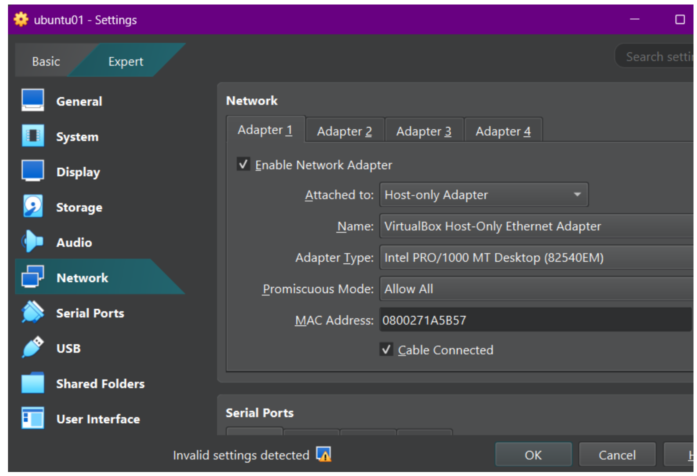
```
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_cod
default qlen 1000
    link/ether 08:00:27:36:14:de brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic eth0
       valid_lft 85537sec preferred_lft 85537sec
```

d. Fix that problem

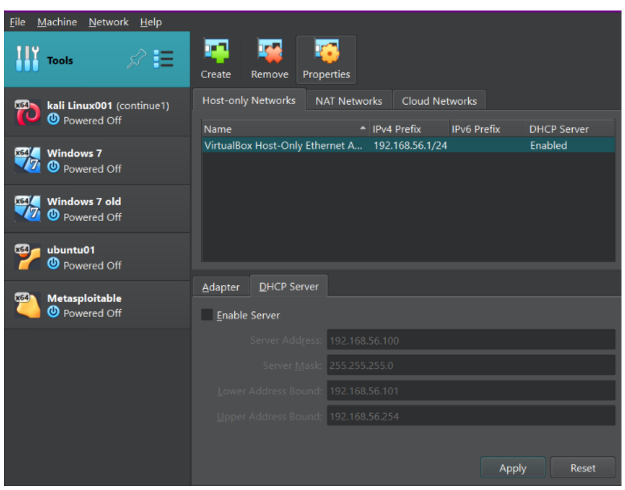
DHCP server wasn't assigning IPs because, The network interface (enp0s3) wasn't properly configured. The DHCP range didn't match the network settings. To fix that follow these steps.

- Put both ubuntu and kali to **Host-Only Network** (not NAT).





- remove tick from "Enable serever"



So, the "Enable Server" checkbox was unchecked in VirtualBox's DHCP settings because it caused conflicts when two DHCP servers were running at the same time. When two servers tried to assign IPs on the same network, devices were not clear on which IP to take, resulting in some devices not getting one in good time. By disabling VirtualBox's DHCP, I ensured only the isc-dhcp-server is used for IP assignment, thus making the network stable and predictable.

- Assigned a Static IP to enp0s3

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo ip addr add 192.160.46.1/24 dev enp0s3
[sudo] password for kaizer:
kaizer@kaizer-VirtualBox:~/Desktop$ sudo ip link set enp0s3 up
```

Now I manually set enp0s3 as the **gateway (**192.160.46.1**)** for the DHCP subnet. This ensures the interface is active and ready to handle DHCP requests.

- Edited DHCP Server Configuration

```
# A slightly different configuration for an internal subnet.
subnet 192.160.46.0 netmask 255.255.255.0 {
  range 192.160.46.100 192.160.46.200;
  option domain-name-servers 8.8.8.8;
  option domain-name "adithya.sliit";
  option subnet-mask 255.255.255.0;
  option routers 192.160.46.1;
  option broadcast-address 10.0.2.255;
  default-lease-time 600;
  max-lease-time 7200;
}
```

- Now again Setting up the client machine

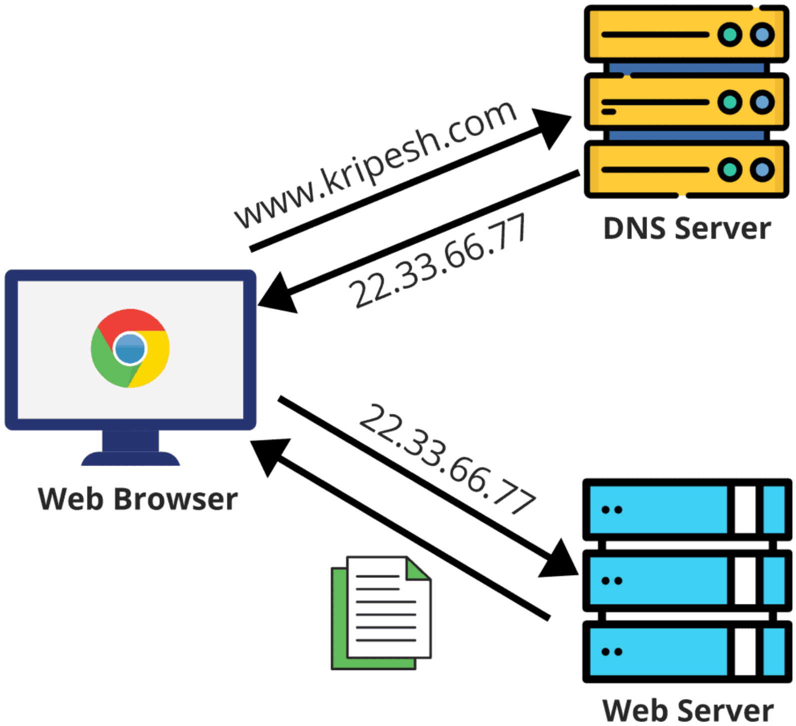
```
┌──(kaizer㉿kali)-[~/Desktop]
└─$ sudo dhclient -r eth0
[sudo] password for kaizer:

┌──(kaizer㉿kali)-[~/Desktop]
└─$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0             UP             fe80::a00:27ff:fe36:14de/64

┌──(kaizer㉿kali)-[~/Desktop]
└─$ sudo dhclient -v eth0
Internet Systems Consortium DHCP Client 4.4.3-P1
```

```
┌──(kaizer㉿kali)-[~/Desktop]
└─$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0             UP             192.160.46.101/24 fe80::a00:27ff:fe36:14de/64
```

Now you can see Kali Linux VM successfully received the IP 192.160.46.101 from configured DHCP server.

DHCP Server is Now Working!

2.2 DNS (Domain Name System)



2.2.1 Introduction to DNS

DNS is a hierarchical naming system that translates human-readable domain names (www.google.com) to machine-readable IP addresses (142.250.190.68). DNS is like the phonebook of the internet.

When you type a website name in your browser:

- Your system queries a DNS resolver.
- The resolver asks authoritative DNS servers where the domain's IP address is.
- The IP address is returned, and your system connects to the server.

2.2.2 Configuring DNS (Local) using BIND9

a. Installing BIND9

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo apt install bind9 bind9utils bind9-doc -y
```

b. Configuring BIND9

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo nano /etc/bind/named.conf.options
```

```
        // If your ISP provided one or more IP addresses for stable
        // nameservers, you probably want to use them as forwarders.
        // Uncomment the following block, and insert the addresses replacing
        // the all-0's placeholder.

         forwarders {
                8.8.8.8;
                8.8.4.4;
         };
```

- Forwarders (like Google's DNS 8.8.8.8) help resolve DNS queries if local DNS server doesn't know the answer.

c. Configuring zone file

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo nano /etc/bind/db.adithyasliit.com
```

```
kaizer@kaizer-VirtualBox: ~/Desktop
  GNU nano 7.2              /etc/bind/db.adithyasliit.com *
$TTL 604800
@       IN SOA ns1.adithyasliit.com. admin.adithyasliit.com. (
                2 ; Serial
                604800 ; Refresh
                86400 ; Retry
                2419200 ; Expire
                604800 ) ; Negative Cache TTL
;
@       IN      NS      ns1.adithyasliit.com.
ns1     IN      A       192.160.41.1
client1 IN      A       192.160.41.101
```

- This configuration takes into account my previously created DHCP server too.

d. **Linking the zone file**

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo nano /etc/bind/named.conf.local
```

```
  GNU nano 7.2                 /etc/bind/named.conf.local
zone "adithyasliit.com" {
        type master;
        file "/etc/bind/db.adithyasliit.com";
};
```

e. **Validating BIND9**

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo named-checkzone adithyasliit.com /etc/bind/db.adithyasliit.com

zone adithyasliit.com/IN: loaded serial 3
OK
```

f. **Restarting and Checking Status**

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo systemctl restart bind9
kaizer@kaizer-VirtualBox:~/Desktop$ sudo systemctl status bind9
● named.service - BIND Domain Name Server
     Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: en>
     Active: active (running) since Wed 2025-05-07 21:33:36 +0530; 18s ago
       Docs: man:named(8)
   Main PID: 4332 (named)
     Status: "running"
      Tasks: 10 (limit: 3750)
```

- Now I also configured my DHCP server to use the DNS server we just created.

Changes

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo nano /etc/dhcp/dhcpd.conf
```

```
# A slightly different configuration for an internal subnet.
subnet 192.160.41.0 netmask 255.255.255.0 {
  range 192.160.41.100 192.160.41.200;
  option domain-name-servers 8.8.8.8;
  option domain-name "adithyasliit.com";
  option subnet-mask 255.255.255.0;
  option routers 192.160.41.1;
  option broadcast-address 192.160.41.255;
  default-lease-time 600;
  max-lease-time 7200;
}
```

- Since I added Google's Public DNS as forwarders when configuring BIND9 it is the same as before. If anything happens with my local DNS server it will automatically switch to google DNS.

g. Verifying DNS resolution on the server

```
kaizer@kaizer-VirtualBox:~/Desktop$ dig client1.adithyasliit.com @192.160.41.1 +short
192.160.41.101
kaizer@kaizer-VirtualBox:~/Desktop$ dig www.adithyasliit.com @192.160.41.1 +short
client1.adithyasliit.com.
192.160.41.101
```

h. Verifying DNS resolution on the client

```
┌──(kaizer㉿kali)-[~/Desktop]
└─$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0             UP             192.160.41.101/24 fe80::a00:27ff:fe36:14de/64

┌──(kaizer㉿kali)-[~/Desktop]
└─$ cat /etc/resolv.conf
domain adithyasliit.com
search adithyasliit.com
nameserver 192.160.41.1
```

- The output confirms that the DNS correctly resolves in the client machine

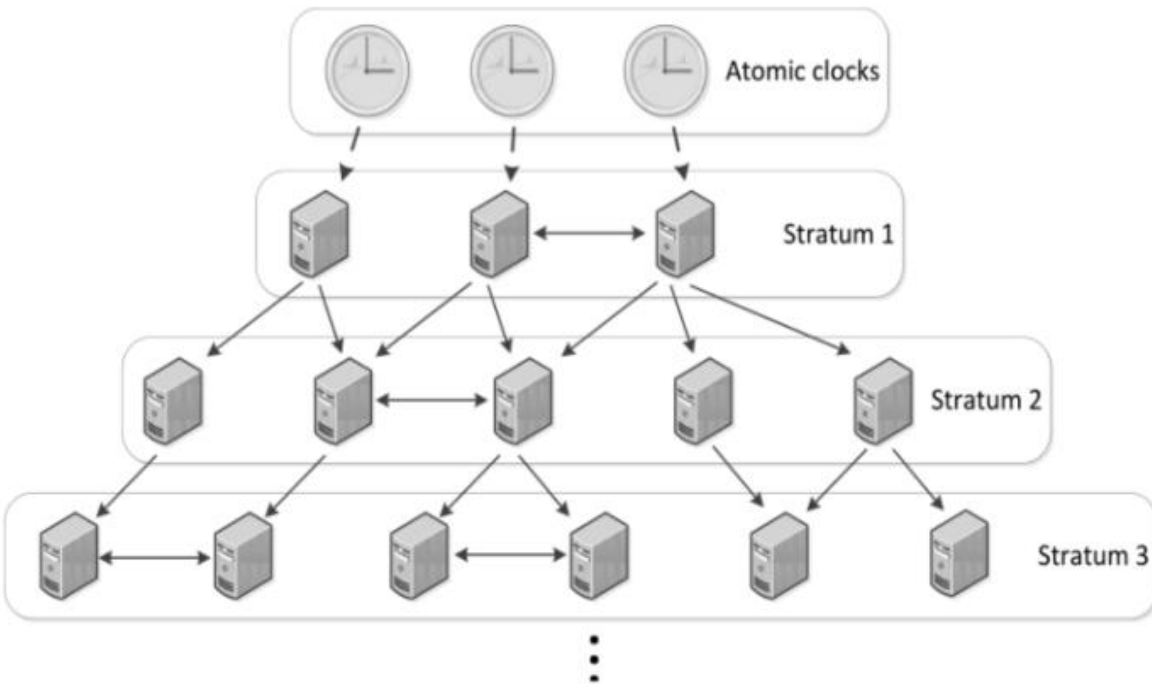
```
┌──(kaizer㉿kali)-[~/Desktop]
└─$ nslookup www.adithyasliit.com
Server:         192.160.41.1
Address:        192.160.41.1#53

www.adithyasliit.com    canonical name = client1.adithyasliit.com.
Name:   client1.adithyasliit.com
Address: 192.160.41.101
```

```
┌──(kaizer㉿kali)-[~/Desktop]
└─$ dig www.adithyasliit.com @192.160.41.1

; <<>> DiG 9.20.7-1-Debian <<>> www.adithyasliit.com @192.160.41.1
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 42155
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 9fb937b545e1dc6301000000681b96c4916b85dd2788ef7c (good)
;; QUESTION SECTION:
;www.adithyasliit.com.          IN      A

;; ANSWER SECTION:
www.adithyasliit.com.   604800  IN      CNAME   client1.adithyasliit.com.
client1.adithyasliit.com. 604800 IN     A       192.160.41.101

;; Query time: 92 msec
;; SERVER: 192.160.41.1#53(192.160.41.1) (UDP)
;; WHEN: Wed May 07 22:52:12 +0530 2025
;; MSG SIZE  rcvd: 115
```

```
─(kaizer㉿kali)-[~/Desktop]
─$ dig client1.adithyasliit.com @192.160.41.1

 <<>> DiG 9.20.7-1-Debian <<>> client1.adithyasliit.com @192.160.41.1
; global options: +cmd
; Got answer:
; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 14261
; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

; OPT PSEUDOSECTION:
 EDNS: version: 0, flags:; udp: 1232
 COOKIE: 2674ae08453f220901000000681b96b0578f75d0affe324c (good)
; QUESTION SECTION:
client1.adithyasliit.com.       IN      A

; ANSWER SECTION:
lient1.adithyasliit.com. 604800 IN      A       192.160.41.101

; Query time: 12 msec
; SERVER: 192.160.41.1#53(192.160.41.1) (UDP)
; WHEN: Wed May 07 22:51:52 +0530 2025
; MSG SIZE  rcvd: 97
```

2.3 NTP (Network Time Protocol)

2.3.1 Introduction



NTP (Network Time Protocol) is a networking protocol used to synchronize the system time of computers with a reliable time source (usually internet-based NTP servers). Accurate system time is critical for security (e.g., logs, certificates), task scheduling (e.g., corn jobs), and communication between systems (e.g., databases, distributed systems).

2.3.2 Configuring NTP server

a. Update the package list & NTP Installing

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo apt update
[sudo] password for kaizer:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://lk.archive.ubuntu.com/ubuntu noble InRelease
```

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo apt install ntp -y
Reading package lists... Done
Building dependency tree... Done
```

b. Configuring NTP

```
#google's NIP server
server time1.google.com iburst
server time2.google.com iburst
server time3.google.com iburst
server time4.google.com iburst

#fallback server
server 0.pool.ntp.org iburst
server 1.pool.ntp.org iburst
```

- For this I first tried running **sudo nano /etc/ntp.conf**, but it only opened an empty file.
- So that, I tried finding the correct file by running
  sudo find /etc -name "ntp.conf*"
- Then I found out the correct file was **/etc/ntpsec/ntp.conf** file.
- Now, I used Google's Public servers for Primary servers and pool.ntp.org project's public NTP servers for fallback.

c. Restarting and Checking status

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo systemctl restart ntp
kaizer@kaizer-VirtualBox:~/Desktop$ sudo systemctl status ntp
● ntpsec.service - Network Time Service
     Loaded: loaded (/usr/lib/systemd/system/ntpsec.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-05-08 03:00:16 +0530; 9s ago
       Docs: man:ntpd(8)
    Process: 5572 ExecStart=/usr/libexec/ntpsec/ntp-systemd-wrapper (code=exited, status=0/SUCCESS)
   Main PID: 5577 (ntpd)
```

d. Verifying synchronization

```
kaizer@kaizer-VirtualBox:~/Desktop$ ntpq -p
     remote                                   refid      st t when poll reach   delay   offset   jitter
=======================================================================================================
 time1.google.com                        .DNS4.          16 u    -   64    0   0.0000   0.0000   0.0001
 time2.google.com                        .DNS4.          16 u    -   64    0   0.0000   0.0000   0.0001
 time3.google.com                        .DNS4.          16 u    -   64    0   0.0000   0.0000   0.0001
 time4.google.com                        .DNS4.          16 u    -   64    0   0.0000   0.0000   0.0001
 time.cloudflare.com                     .DNS4.          16 u    -   64    0   0.0000   0.0000   0.0001
 time.cloudflare.com                     .DNS4.          16 u    -   64    0   0.0000   0.0000   0.0001
```

- The output confirms that NTP service is working as intended.

```
kaizer@kaizer-VirtualBox:~/Desktop$ timedatectl
               Local time: Thu 2025-05-08 03:03:43 +0530
           Universal time: Wed 2025-05-07 21:33:43 UTC
                 RTC time: Wed 2025-05-07 21:33:43
                Time zone: Asia/Colombo (+0530, +0530)
System clock synchronized: yes
              NTP service: n/a
          RTC in local TZ: no
```

3. Security and Other servers

3.1 Shell Scripting

3.1.1 Shell script to automate log cleanup and archival

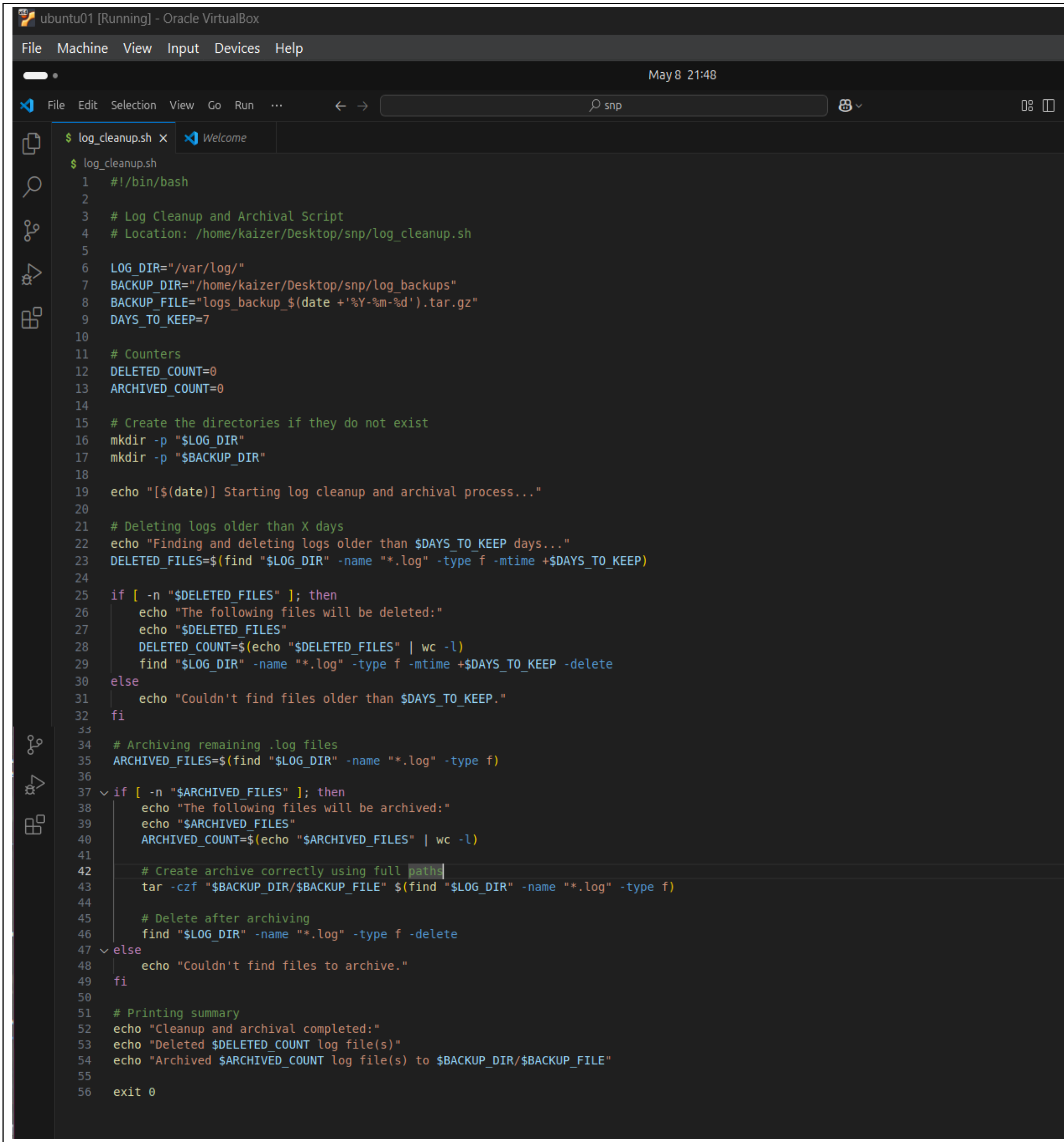
a. Create a directory for backups

```
kaizer@kaizer-VirtualBox:~/Desktop/snp$ mkdir log_backups
kaizer@kaizer-VirtualBox:~/Desktop/snp$ ls
log_backups
```

b. Create the shell script

```
kaizer@kaizer-VirtualBox:~/Desktop/snp$ code log_cleanup.sh
```

- I use VS Code because it is easier to use than nano. That’s why I use this command.
- I put Code on the next page.




```bash
#!/bin/bash

# Log Cleanup and Archival Script
# Location: /home/kaizer/Desktop/snp/log_cleanup.sh

LOG_DIR="/var/log/"
BACKUP_DIR="/home/kaizer/Desktop/snp/log_backups"
BACKUP_FILE="logs_backup_$(date +'%Y-%m-%d').tar.gz"
DAYS_TO_KEEP=7

# Counters
DELETED_COUNT=0
ARCHIVED_COUNT=0

# Create the directories if they do not exist
mkdir -p "$LOG_DIR"
mkdir -p "$BACKUP_DIR"

echo "[$(date)] Starting log cleanup and archival process..."

# Deleting logs older than X days
echo "Finding and deleting logs older than $DAYS_TO_KEEP days..."
DELETED_FILES=$(find "$LOG_DIR" -name "*.log" -type f -mtime +$DAYS_TO_KEEP)

if [ -n "$DELETED_FILES" ]; then
    echo "The following files will be deleted:"
    echo "$DELETED_FILES"
    DELETED_COUNT=$(echo "$DELETED_FILES" | wc -l)
    find "$LOG_DIR" -name "*.log" -type f -mtime +$DAYS_TO_KEEP -delete
else
    echo "Couldn't find files older than $DAYS_TO_KEEP."
fi

# Archiving remaining .log files
ARCHIVED_FILES=$(find "$LOG_DIR" -name "*.log" -type f)

if [ -n "$ARCHIVED_FILES" ]; then
    echo "The following files will be archived:"
    echo "$ARCHIVED_FILES"
    ARCHIVED_COUNT=$(echo "$ARCHIVED_FILES" | wc -l)

    # Create archive correctly using full paths
    tar -czf "$BACKUP_DIR/$BACKUP_FILE" $(find "$LOG_DIR" -name "*.log" -type f)

    # Delete after archiving
    find "$LOG_DIR" -name "*.log" -type f -delete
else
    echo "Couldn't find files to archive."
fi

# Printing summary
echo "Cleanup and archival completed:"
echo "Deleted $DELETED_COUNT log file(s)"
echo "Archived $ARCHIVED_COUNT log file(s) to $BACKUP_DIR/$BACKUP_FILE"

exit 0
```

c. Make the script executable

```
kaizer@kaizer-VirtualBox:~/Desktop/snp$ chmod +x log_cleanup.sh
```

```
kaizer@kaizer-VirtualBox:~/Desktop/snp$ ls
log_backups   log_cleanup.sh
```

d. Script Testing

```
kaizer@kaizer-VirtualBox:~/Desktop/snp$ sudo ./log_cleanup.sh
[sudo] password for kaizer:
[Thu May  8 09:40:26 PM +0530 2025] Starting log cleanup and archival proc
ess...
Finding and deleting logs older than 7 days...
Couldn't find files older than 7.
The following files will be archived:
/var/log/kern.log
/var/log/apport.log
/var/log/auth.log
/var/log/unattended-upgrades/unattended-upgrades-shutdown.log
/var/log/boot.log
/var/log/gpu-manager.log
tar: Removing leading `/' from member names
tar: Removing leading `/' from hard link targets
Cleanup and archival completed:
Deleted 0 log file(s)
Archived 6 log file(s) to /home/kaizer/Desktop/snp/log_backups/logs_backup
_2025-05-08.tar.gz
```

- The output shows that the script works as intended and we can find the compressed file in the appropriate location

```
kaizer@kaizer-VirtualBox:~/Desktop/snp$ cd log_backups
kaizer@kaizer-VirtualBox:~/Desktop/snp/log_backups$ ls
logs_backup_2025-05-08.tar.gz
kaizer@kaizer-VirtualBox:~/Desktop/snp/log_backups$
```

e. Cron Job Setup

cron job is a scheduled task that is in Linux and that runs automatically at specified times or intervals. we use it to automate repetitive tasks like running your log cleanup script every week.

```
kaizer@kaizer-VirtualBox:~/Desktop/snp$ crontab -e
```

```
Select an editor.  To change later, run 'select-editor'.
  1. /bin/nano        <---- easiest
  2. /usr/bin/vim.tiny
  3. /bin/ed

Choose 1-3 [1]: 1
crontab: installing new crontab
```

add this line to the bottom of the opened file

```
0 0 * * 0 /bin/bash /home/kaizer/Desktop/snp/log_cleanup.sh
```

this use to run the **log_cleanup.sh** script automatically every Sunday at 12:00 A.M .

3.2 SSH (Secure Shell)

a. Installing SSH

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo apt install openssh-server -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openssh-server is already the newest version (1:9.6p1-3ubuntu13.11).
The following package was automatically installed and is no longer required:
  libllvm17t64
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 271 not upgraded.
```

b. Enabling SSH Service

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo systemctl enable --now ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/system
d/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
```

c. Checking SSH Status

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enab>
     Active: active (running) since Fri 2025-05-09 09:12:33 +0530; 13min ago
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
   Main PID: 5511 (sshd)
      Tasks: 1 (limit: 3750)
     Memory: 1.2M (peak: 4.1M)
```

- Shows it was enabled successfully

d. Connecting through a client

```
┌──(kaizer㉿kali)-[~]
└─$ ssh kaizer@192.160.41.1
The authenticity of host '192.160.41.1 (192.160.41.1)' can't be established.
ED25519 key fingerprint is SHA256:pV9cmAF+7RlOqrYSCqsMvmjn0BR4uKbj7mhcHhhmxEQ.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.160.41.1' (ED25519) to the list of known hosts.
kaizer@192.160.41.1's password:
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.11.0-25-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

265 updates can be applied immediately.
33 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

10 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

```
kaizer@kaizer-VirtualBox:~$ ls
Desktop    Downloads  Music     Public  snp  Templates  Videos
Documents  folder1    Pictures  snap    SOS  testFile
kaizer@kaizer-VirtualBox:~$ cd snp
kaizer@kaizer-VirtualBox:~/snp$ ls
'IT23622678_-_IE2012_SNP_Assignment[1].pdf'   LAB4   shell   'vs code'
kaizer@kaizer-VirtualBox:~/snp$
```

3.3 iptables (Firewall rules)

a. Clearing existing tables

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo iptables -F
[sudo] password for kaizer:
```

b. Blocking social media (Facebook, Instagram, Twitter)

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo iptables -A OUTPUT -p tcp -d instagram.com -j DROP

kaizer@kaizer-VirtualBox:~/Desktop$ sudo iptables -A OUTPUT -p tcp -d facebook.com -j DROP

kaizer@kaizer-VirtualBox:~/Desktop$ sudo iptables -A OUTPUT -p tcp -d twitter.com -j DROP
```

- **-A OUTPUT:** Applies to outgoing traffic
- **-p tcp:** Blocks TCP connections
- **-d facebook.com:** Target domain
- **-j DROP:** Discards packets

c. Allow HTTPS but block HTTP

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo iptables -A OUTPUT -p tcp --dport 80 -j DROP
kaizer@kaizer-VirtualBox:~/Desktop$ sudo iptables -A OUTPUT -p tcp --dport 443 -j ACCEPT
```

- **-dport 80:** HTTP port
- **-j ACCEPT:** Accepts packets
- **-dport 443:** HTTPS port

d. Verifying rules

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo iptables -L -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 161 packets, 107K bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 DROP       tcp  --  any    any     anywhere             instagram-p42-shv-01-sin11.fbcdn.net
    0     0 DROP       tcp  --  any    any     anywhere             edge-star-mini-shv-01-sin11.facebook.com
    0     0 DROP       tcp  --  any    any     anywhere             162.159.140.229
    0     0 DROP       tcp  --  any    any     anywhere             anywhere             tcp dpt:http
    2   120 ACCEPT     tcp  --  any    any     anywhere             anywhere             tcp dpt:https
kaizer@kaizer-VirtualBox:~/Desktop$
```

3.4 Web Server (Apache)

a. Installing Apache

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo apt install apache2 -y
[sudo] password for kaizer:
Reading package lists... Done
```

b. Checking the web root directory

```
kaizer@kaizer-VirtualBox:~/Desktop$ ls /var/www/html
index.html
```

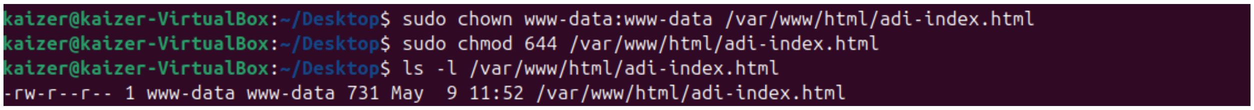
There already is a default index.html file by Apache, Then I will create another file for this implementation

c. Creating a separate personal file & Writing the HTML code

```
kaizer@kaizer-VirtualBox:~/Desktop$ sudo nano  /var/www/html/adi-index.html
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>SNP - Assignment</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            line-height: 1.6;
            margin: 20px;
        }
        h1 {
            color: #333;
            border-bottom: 1px solid #eee;
            padding-bottom: 10px;
        }
        .student-info {
            margin-top: 20px;
            font-weight: bold;
        }
    </style>
</head>
<body>
    <h1>SNP - Assignment</h1>

    <div class="student-info">
        <p>H.A.№ A HETПACHOHI</p>
        <p>IT23605398</p>
    </div>
</body>
</html>
```

d. Setting up correct permissions



• In the first command the ownership of the sanila-index.html file is given to www.data user.

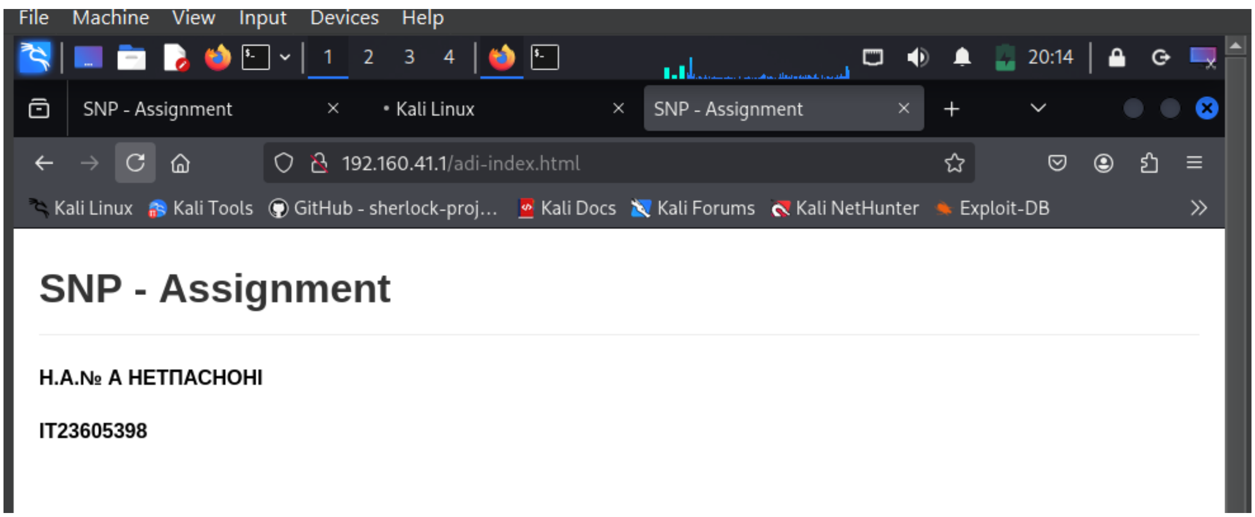
• Because Apache runs as www.data user.

• In the second command sets file permissions to:

- 6 (Owner: Read + Write)
- 4 (Group: Read-only)
- (Others: Read-only)

We can further confirm that by running ls command

e. Accessing the server using a client machine



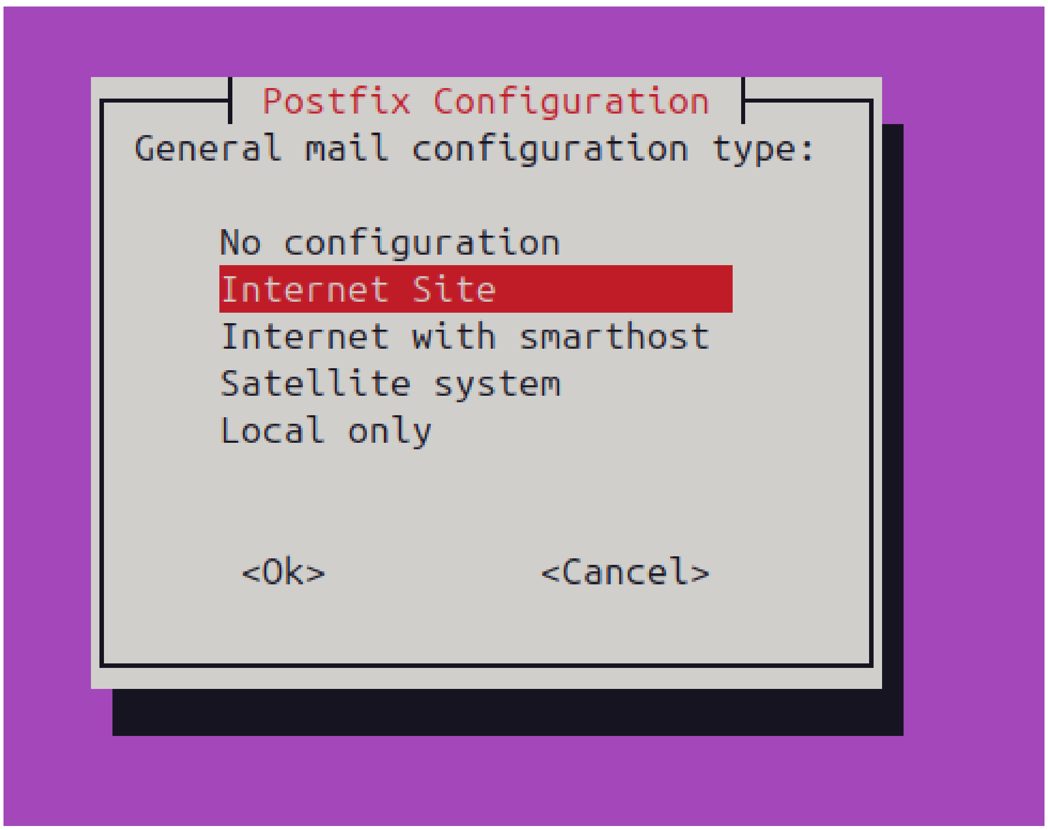
You can see access the server by typing the server IP address and the file in the web root directory in the address bar of your preferred browser.

3.5 Email Server (Postfix)

a. Installing Postfix

```
kaizer@kaizer-VirtualBox:~$ sudo apt install postfix -y
Reading package lists... Done
Building dependency tree... Done
```

- The installation process a prompt will pop up
- Select Internet site from that list of options then hit enter



- In the next prompt I enter my hostname.

b. Testing the Email System

Sending a Test Email

```
kaizer@kaizer-VirtualBox:~$ echo 'test mail' | mail -s 'Test Subject' kaizer
```

- A test email with the subject "Test Subject" is sent to the local user kaizer.

Checking the Mailbox

```
kaizer@kaizer-VirtualBox:~$ mail
```

Out put

```
"/var/mail/kaizer": 3 messages 3 new
>N   1 Mail Delivery Syst Fri May  9 12:45  83/3105  Undelivered Mail Returned
 N   2 Mail Delivery Syst Fri May  9 12:47  83/3120  Undelivered Mail Returned
 N   3 kaizer             Fri May  9 13:19  13/452   Test Subject
?
```

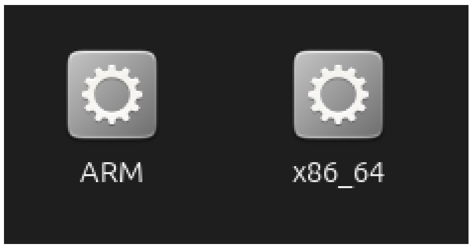
1 and 2 are two mistakes made by me but, third one is successful test email (works because it’s local).

4. Linux GDB

4.1 Execution Process

a. Check System Architecture

```
kaizer@kaizer-VirtualBox:~/Desktop/snp$ uname -m
x86_64
```



- The executable file contained two files that are meant for two different system architectures.

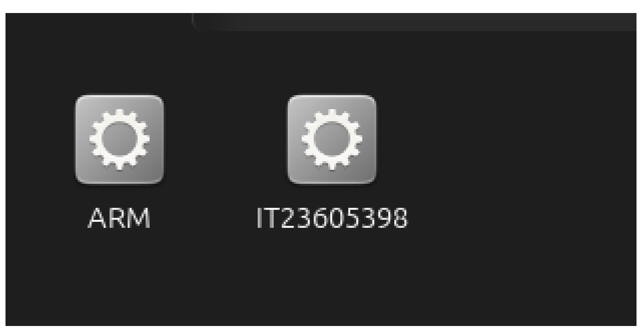
b. Run the Selected Executable

- The x86_64 was chosen as it matched the system architecture. After granting permission, the program was executed.

```
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$ chmod +x ./x86_64
```

```
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$ ./x86_64
Enter the student IT number: IT23605398
```

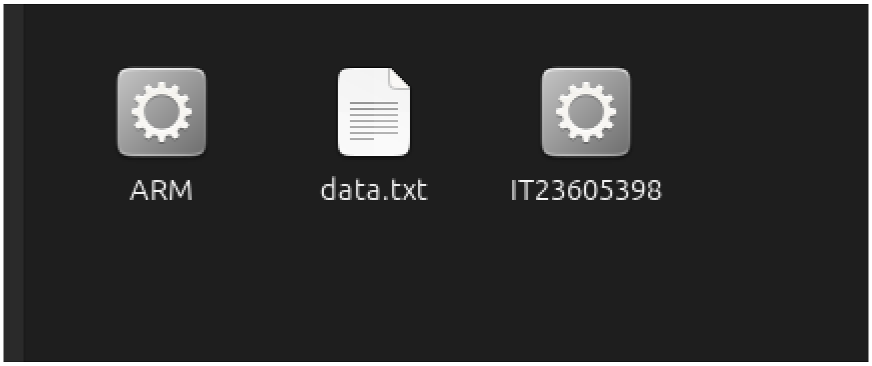
- The program asked for the registration number, and when provided, the program renamed it as the registration number.



c. Analyze the Generated Executable

The program, now renamed, was executed again. This created a text file.

```
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$ ./IT23605398
[sudo] password for kaizer:
```




```
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$ nano data.txt
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$ ls -lt
total 96
-rwxrwxr-x 1 kaizer kaizer    22 May  9 22:37 data.txt
-rwxrwxr-x 1 kaizer kaizer 20368 May  9 22:34 IT23605398
-rw-rw-r-- 1 kaizer kaizer 70824 Mar 17 06:26 ARM
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$ ./IT23605398
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$ ls -lt
total 96
-rwxrwxr-x 1 kaizer kaizer    22 May  9 22:43 data.txt
-rwxrwxr-x 1 kaizer kaizer 20368 May  9 22:34 IT23605398
-rw-rw-r-- 1 kaizer kaizer 70824 Mar 17 06:26 ARM
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$ stat data.txt
  File: data.txt
  Size: 22              Blocks: 8          IO Block: 4096   regular file
Device: 8,2     Inode: 813708      Links: 1
Access: (0775/-rwxrwxr-x)  Uid: ( 1000/  kaizer)   Gid: ( 1000/  kaizer)
Access: 2025-05-09 22:39:25.750052964 +0530
Modify: 2025-05-09 22:43:42.154060202 +0530
Change: 2025-05-09 22:43:42.154060202 +0530
 Birth: 2025-05-09 22:36:43.659048388 +0530
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$
```

The creation time confirms that it was created after the execution of the aforementioned program.

4.2 Debugging Process

The first step was to open GDB with the executable file. -q was used for smoother output.

```
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$ gdb -q IT2360539
8
Reading symbols from IT23605398...
(gdb)
```

```
(gdb) info functions
All defined functions:

File IT23605398.c:
13:     int main();
5:      void xor_encrypt_decrypt(char *, const char *);

Non-debugging symbols:
0x0000000000001000  _init
0x00000000000010d0  __cxa_finalize@plt
0x00000000000010e0  puts@plt
0x00000000000010f0  fclose@plt
0x0000000000001100  strlen@plt
0x0000000000001110  __stack_chk_fail@plt
0x0000000000001120  pclose@plt
0x0000000000001130  fputs@plt
0x0000000000001140  strcspn@plt
0x0000000000001150  fgets@plt
0x0000000000001160  popen@plt
0x0000000000001170  fopen@plt
0x0000000000001180  _start
0x00000000000011b0  deregister_tm_clones
0x00000000000011e0  register_tm_clones
0x0000000000001220  __do_global_dtors_aux
--Type <RET> for more, q to quit, c to continue without paging--
```

The “info functions” command was used to identify the functions in this particular program.

This shows that there are two functions in the program. The first one was the common main function, but the second one was named as “xor_encrypt_decrypt”. This means the operation in this program is encryption. There are two parameters that are being passed. Both of them are passing as characters, and one parameter is set as a constant. This might be a key that is used for encryption.

Now the disassembled language was set to Intel because by default it is set to AT&T, and Intel is more user-friendly.

```
(gdb) set disassembly-flavor intel
```

Next, the disassembled code was taken from both functions to get a rough idea of the program.

```
(gdb) set disassembly-flavor intel
(gdb) disas main
Dump of assembler code for function main:
   0x00000000000012f0 <+0>:     endbr64
   0x00000000000012f4 <+4>:     push   rbp
   0x00000000000012f5 <+5>:     mov    rbp,rsp
   0x00000000000012f8 <+8>:     sub    rsp,0x60
   0x00000000000012fc <+12>:    mov    rax,QWORD PTR fs:0x28
   0x0000000000001305 <+21>:    mov    QWORD PTR [rbp-0x8],rax
   0x0000000000001309 <+25>:    xor    eax,eax
   0x000000000000130b <+27>:    lea    rax,[rip+0xcf6]        # 0x2008
   0x0000000000001312 <+34>:    mov    rsi,rax
   0x0000000000001315 <+37>:    lea    rax,[rip+0xcf4]        # 0x2010
   0x000000000000131c <+44>:    mov    rdi,rax
   0x000000000000131f <+47>:    call   0x1160 <popen@plt>
   0x0000000000001324 <+52>:    mov    QWORD PTR [rbp-0x58],rax
   0x0000000000001328 <+56>:    cmp    QWORD PTR [rbp-0x58],0x0
   0x000000000000132d <+61>:    jne    0x1348 <main+88>
   0x000000000000132f <+63>:    lea    rax,[rip+0xd02]        # 0x2038
   0x0000000000001336 <+70>:    mov    rdi,rax
   0x0000000000001339 <+73>:    call   0x10e0 <puts@plt>
   0x000000000000133e <+78>:    mov    eax,0x1
   0x0000000000001343 <+83>:    jmp    0x1400 <main+272>
   0x0000000000001348 <+88>:    mov    rdx,QWORD PTR [rbp-0x58]
   0x000000000000134c <+92>:    lea    rax,[rbp-0x40]
   0x0000000000001350 <+96>:    mov    esi,0x32
   0x0000000000001355 <+101>:   mov    rdi,rax
   0x0000000000001358 <+104>:   call   0x1150 <fgets@plt>
   0x000000000000135d <+109>:   mov    rax,QWORD PTR [rbp-0x58]
   0x0000000000001361 <+113>:   mov    rdi,rax
   0x0000000000001364 <+116>:   call   0x1120 <pclose@plt>
   0x0000000000001369 <+121>:   lea    rax,[rbp-0x40]
   0x000000000000136d <+125>:   lea    rdx,[rip+0xcda]        # 0x204e
   0x0000000000001374 <+132>:   mov    rsi,rdx
   0x0000000000001377 <+135>:   mov    rdi,rax
   0x000000000000137a <+138>:   call   0x1140 <strcspn@plt>
   0x000000000000137f <+143>:   mov    BYTE PTR [rbp+rax*1-0x40],0x0
   0x0000000000001384 <+148>:   lea    rax,[rip+0xcc5]        # 0x2050
   0x000000000000138b <+155>:   mov    QWORD PTR [rbp-0x50],rax
--Type <RET> for more, q to quit, c to continue without paging--<RET>
   0x000000000000138f <+159>:   mov    rdx,QWORD PTR [rbp-0x50]
   0x0000000000001393 <+163>:   lea    rax,[rbp-0x40]
   0x0000000000001397 <+167>:   mov    rsi,rdx
   0x000000000000139a <+170>:   mov    rdi,rax
   0x000000000000139d <+173>:   call   0x1269 <xor_encrypt_decrypt>
   0x00000000000013a2 <+178>:   lea    rax,[rip+0xcab]        # 0x2054
   0x00000000000013a9 <+185>:   mov    rsi,rax
   0x00000000000013ac <+188>:   lea    rax,[rip+0xca3]        # 0x2056
   0x00000000000013b3 <+195>:   mov    rdi,rax
   0x00000000000013b6 <+198>:   call   0x1170 <fopen@plt>
   0x00000000000013bb <+203>:   mov    QWORD PTR [rbp-0x48],rax
   0x00000000000013bf <+207>:   cmp    QWORD PTR [rbp-0x48],0x0
   0x00000000000013c4 <+212>:   jne    0x13dc <main+236>
   0x00000000000013c6 <+214>:   lea    rax,[rip+0xc92]        # 0x205f
   0x00000000000013cd <+221>:   mov    rdi,rax
   0x00000000000013d0 <+224>:   call   0x10e0 <puts@plt>
   0x00000000000013d5 <+229>:   mov    eax,0x1
   0x00000000000013da <+234>:   jmp    0x1400 <main+272>
   0x00000000000013dc <+236>:   mov    rdx,QWORD PTR [rbp-0x48]
   0x00000000000013e0 <+240>:   lea    rax,[rbp-0x40]
   0x00000000000013e4 <+244>:   mov    rsi,rdx
   0x00000000000013e7 <+247>:   mov    rdi,rax
   0x00000000000013ea <+250>:   call   0x1130 <fputs@plt>
   0x00000000000013ef <+255>:   mov    rax,QWORD PTR [rbp-0x48]
   0x00000000000013f3 <+259>:   mov    rdi,rax
   0x00000000000013f6 <+262>:   call   0x10f0 <fclose@plt>
   0x00000000000013fb <+267>:   mov    eax,0x0
   0x0000000000001400 <+272>:   mov    rdx,QWORD PTR [rbp-0x8]
   0x0000000000001404 <+276>:   sub    rdx,QWORD PTR fs:0x28
   0x000000000000140d <+285>:   je     0x1414 <main+292>
   0x000000000000140f <+287>:   call   0x1110 <__stack_chk_fail@plt>
   0x0000000000001414 <+292>:   leave
   0x0000000000001415 <+293>:   ret
End of assembler dump.
(gdb)
```

```
(gdb) disas xor_encrypt_decrypt
Dump of assembler code for function xor_encrypt_decrypt:
   0x0000000000001269 <+0>:     endbr64
   0x000000000000126d <+4>:     push   rbp
   0x000000000000126e <+5>:     mov    rbp,rsp
   0x0000000000001271 <+8>:     sub    rsp,0x30
   0x0000000000001275 <+12>:    mov    QWORD PTR [rbp-0x28],rdi
   0x0000000000001279 <+16>:    mov    QWORD PTR [rbp-0x30],rsi
   0x000000000000127d <+20>:    mov    rax,QWORD PTR [rbp-0x28]
   0x0000000000001281 <+24>:    mov    rdi,rax
   0x0000000000001284 <+27>:    call   0x1100 <strlen@plt>
   0x0000000000001289 <+32>:    mov    QWORD PTR [rbp-0x10],rax
   0x000000000000128d <+36>:    mov    rax,QWORD PTR [rbp-0x30]
   0x0000000000001291 <+40>:    mov    rdi,rax
   0x0000000000001294 <+43>:    call   0x1100 <strlen@plt>
   0x0000000000001299 <+48>:    mov    QWORD PTR [rbp-0x8],rax
   0x000000000000129d <+52>:    mov    QWORD PTR [rbp-0x18],0x0
   0x00000000000012a5 <+60>:    jmp    0x12e2 <xor_encrypt_decrypt+121>
   0x00000000000012a7 <+62>:    mov    rdx,QWORD PTR [rbp-0x28]
   0x00000000000012ab <+66>:    mov    rax,QWORD PTR [rbp-0x18]
   0x00000000000012af <+70>:    add    rax,rdx
   0x00000000000012b2 <+73>:    movzx  esi,BYTE PTR [rax]
   0x00000000000012b5 <+76>:    mov    rax,QWORD PTR [rbp-0x18]
   0x00000000000012b9 <+80>:    mov    edx,0x0
   0x00000000000012be <+85>:    div    QWORD PTR [rbp-0x8]
   0x00000000000012c2 <+89>:    mov    rax,QWORD PTR [rbp-0x30]
   0x00000000000012c6 <+93>:    add    rax,rdx
   0x00000000000012c9 <+96>:    movzx  ecx,BYTE PTR [rax]
   0x00000000000012cc <+99>:    mov    rdx,QWORD PTR [rbp-0x28]
   0x00000000000012d0 <+103>:   mov    rax,QWORD PTR [rbp-0x18]
   0x00000000000012d4 <+107>:   add    rax,rdx
   0x00000000000012d7 <+110>:   xor    esi,ecx
   0x00000000000012d9 <+112>:   mov    edx,esi
   0x00000000000012db <+114>:   mov    BYTE PTR [rax],dl
   0x00000000000012dd <+116>:   add    QWORD PTR [rbp-0x18],0x1
   0x00000000000012e2 <+121>:   mov    rax,QWORD PTR [rbp-0x18]
   0x00000000000012e6 <+125>:   cmp    rax,QWORD PTR [rbp-0x10]
   0x00000000000012ea <+129>:   jb     0x12a7 <xor_encrypt_decrypt+62>
   0x00000000000012ec <+131>:   nop
   0x00000000000012ed <+132>:   nop
   0x00000000000012ee <+133>:   leave
   0x00000000000012ef <+134>:   ret
End of assembler dump.
```

In the summary,

- The main program calls popen() with a command and captures its output,
- Reads a line from the output into a buffer,
- Calls strcspn() to trim newline characters,
- Calls xor_encrypt_decrypt() function on the buffer,
- Opens a file for writing (fopen),
- Writes the buffer into this file using fputs.
- Then closes the program.

- It was assumed that the file being created was the data.txt file that was created earlier. Which means it must contain whatever data was encrypted.

- Then, breakpoints were set for both functions to analyze their behaviors.

```
(gdb) break main
Note: breakpoint 1 also set at pc 0x12fc.
Breakpoint 2 at 0x12fc: file IT23605398.c, line 13.
(gdb) break xor_encrypt_decrypt
Breakpoint 3 at 0x127d: file IT23605398.c, line 6.
(gdb)
```

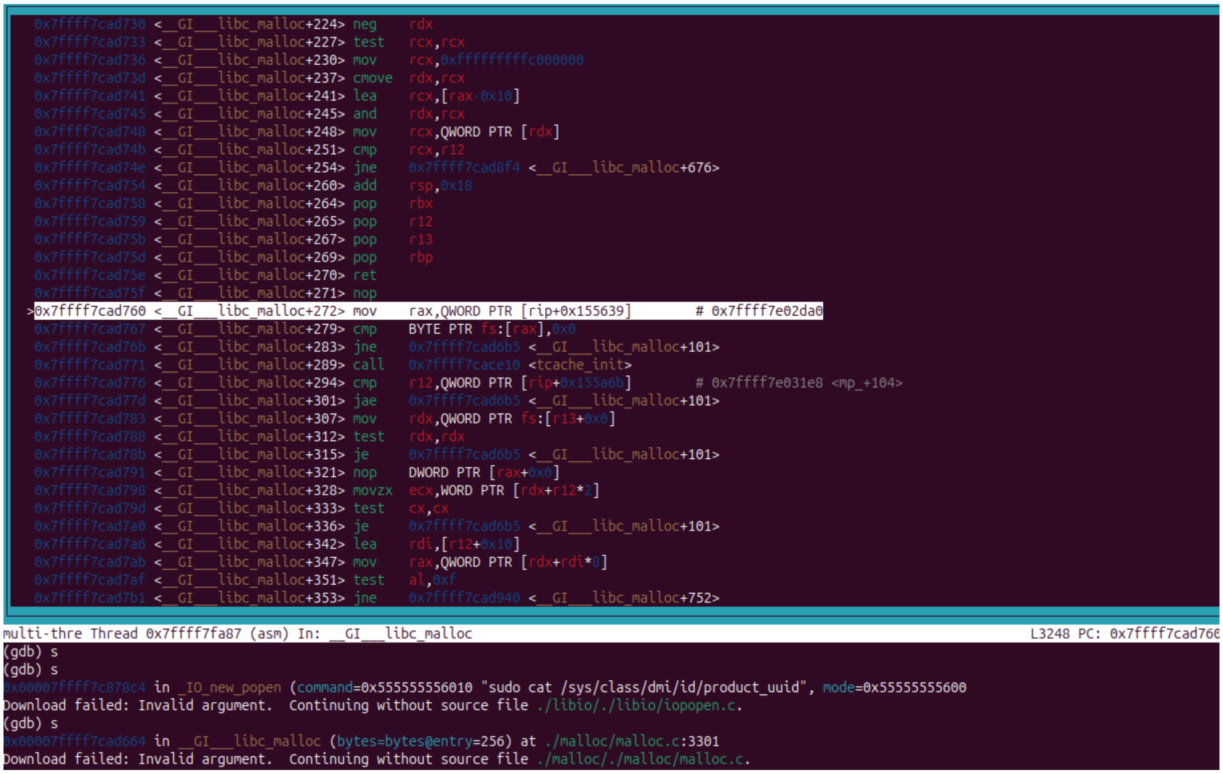
- Then, the program was executed with the run command. As set it was stopped at the main function.

```
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kaizer/Desktop/snp/Executables/Executables/IT23605398
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at IT23605398.c:13
13      in IT23605398.c
```

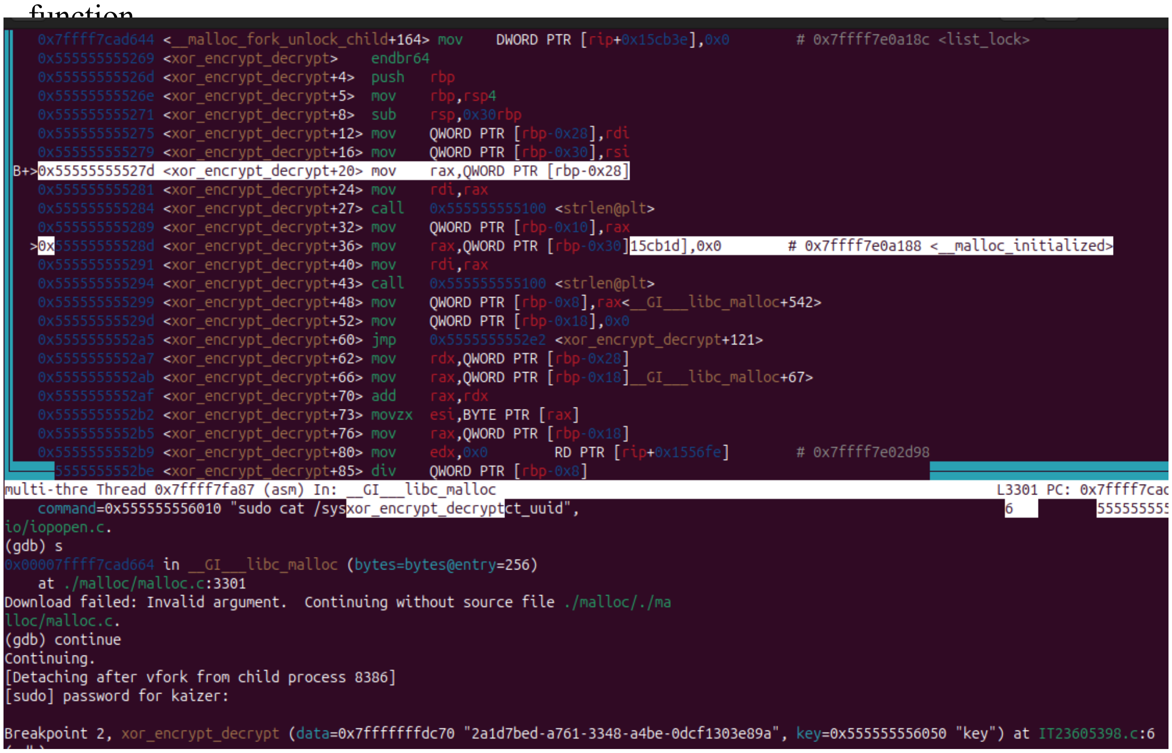
```
(gdb) layout asm
```

- Then "**layout asm**" was used to see the assembly code while the program was running.

- While using the "**step**" command, a command was shown to read a file. The program is about to execute the command "**sudo cat /sys/class/dmi/id/product_uuid**" using "**popen()**" in read mode. The assembly is part of the C library's implementation of popen, handling memory allocation.



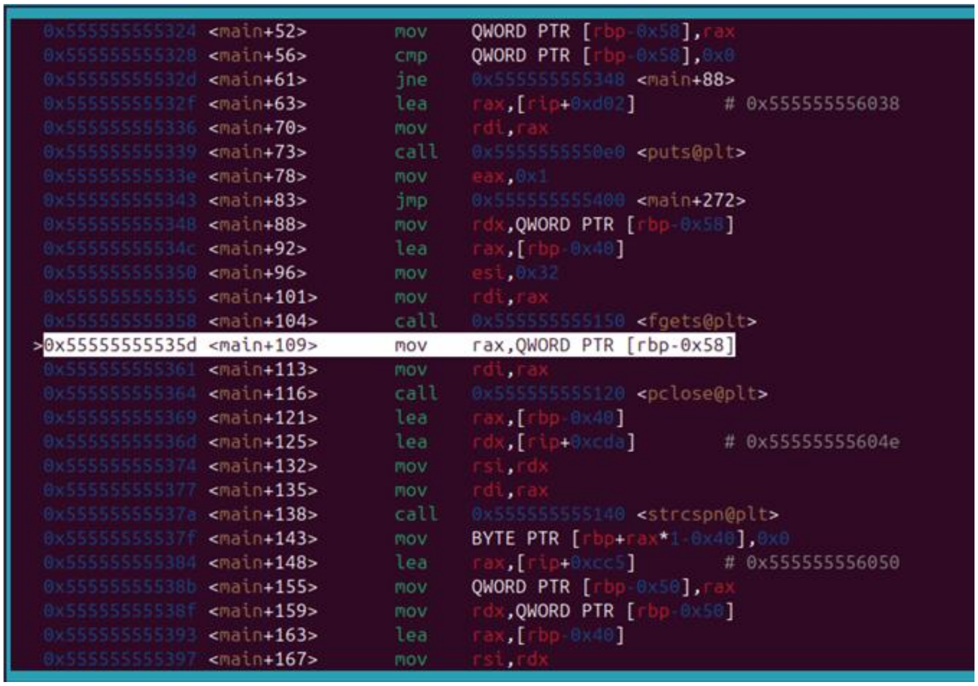
- -After that "**continue**" command was given to continue the program until the next break point. The program stopped at the second breakpoint point which was set at the XOR function.



- At the second breakpoint, it was noticed that two values were passed as parameters.

- “**3f5a8a95-69d6-3647-83d5-91c875414b4f** ” value was passed as the first parameter and “**key**” was passed as the second parameter. Which means the first value must be encrypted by the “key” keyword and the result must be the output of the .txt file. It is safe to assume that the first value was taken from the file shown before, with the path.

- The value was also caught while it was loading into the buffer.



So to conclude, the program reads a value from the discovered command, and then uses the value that is returned and a keyword to encrypt this value using the XOR function, and outputs the encrypted value to the data.txt file.

4.3 File System Analysis

a. Check File Type

Using the file command the type of content the file stores was discovered.

```
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$ file data.txt
data.txt: data
```

b. Adjust Permissions for Inspection

The next step was to see the inside of the file using a text editor. To do that, the file needed a permission change.

```
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$ chmod 700 data.txt
```

C. File Contents

```
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$ nano data.txt
```

```
  GNU nano 7.2
Y^DH^OR^[^N^AT
ROZHJXQAF^DM
```

It was shown as some random data. Probably encrypted, since in the debugging part, it was discovered that the file might hold encrypted data. After a few executions of the IT numbered program, the values stayed the same.

d. Hexdump Analysis

```
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$ strings data.txt
```

```
kaizer@kaizer-VirtualBox:~/Desktop/snp/Executables/Executables$ hexdump data.txt
0000000 0459 0f48 1b52 010e 0a54 4f52 485a 584a
0000010 4151 0446 094d
0000016
```

The **hexdump** command was also used to find meaningful content. But it displays the file byte by-byte in ASCII, using escape sequences for non-printable characters.

In the current state. It was impossible to get an idea about the content of the data.txt file. A decryption method was suggested to check whether it provided the original value that was discovered in the debugging part.

So, a C language program and a copy of the .txt file were used to decrypt the data.txt file.

kaizer@kaizer-VirtualBox: ~/Desktop/snp/code

GNU nano 7.2 1code.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void xor_encrypt_decrypt(char *data, size_t len, const char *key) {
    size_t key_len = strlen(key);
    for (size_t i = 0; i < len; i++) {
        data[i] ^= key[i % key_len];
    }
}

int main() {
    FILE *fp = fopen("data.txt", "rb");
    if (!fp) {
        perror("fopen failed");
        return 1;
    }

    size_t uuid_len;
    if (fread(&uuid_len, sizeof(size_t), 1, fp) != 1) {
        perror("fread length failed");
        fclose(fp);
        return 1;
    }

    char *buffer = malloc(uuid_len + 1);
    if (!buffer) {
        perror("malloc failed");
        fclose(fp);
        return 1;
    }

    if (fread(buffer, 1, uuid_len, fp) != uuid_len) {
        perror("fread data failed");
        free(buffer);
        fclose(fp);
        return 1;
    }
    fclose(fp);

    xor_encrypt_decrypt(buffer, uuid_len, "key");
    buffer[uuid_len] = '\0';  // Null-terminate for printing

    printf("Decrypted UUID: %s\n", buffer);
    free(buffer);
    return 0;
}
```

```
kaizer@kaizer-VirtualBox:~/Desktop/snp/code$ ./1code
Decrypted UUID: 2a1d7bed-a761-3348-a4be-0dcf1303e89a
```

The resulting decrypted.txt we saw before in the debugging part.,

```
(gdb) continue
Continuing.
[Detaching after vfork from child process 8386]
[sudo] password for kaizer:

Breakpoint 2, xor_encrypt_decrypt (data=0x7fffffffdc70 "2a1d7bed-a761-3348-a4be-0dcf1303e89a", key=0x555555556050 "key") at IT23605398.c:6
```

This confirms that the data.txt file is the result of the executable program and that it contains the XOR-encrypted value of the discovered value.

Conclusion

This report documents in detail the set-up and configuration of Linux environments and some important network services, thus demonstrating practical skills in system administration and networking. The major activities included setting up a working VM; setting up the DHCP and DNS servers to manage networks automatically; and, ensuring time synchronization across networks via NTP. Other activities included securing the network using SSH and iptables and setting up automatic cleaning of logs; deploying the Apache and Postfix servers. The debugging and file analysis part stretched the problem-solving skills by decrypting an XOR-encrypted file. In essence, this project indicates a good grasp of Linux systems and network services and provides a clear and easy-to-replicate reference for later use. The methodical layout and explanations make it a great resource for someone wanting to learn or apply it in practice.

End.